

TDS200 · Kryssplattform

Session 7

Native Device Features & Authentication

Camera · Image Gallery · Progress Bar · Firebase Authentication

Cross-Platform Mobile Development · Lecturer: Yuan Lin

What you'll learn today

- Capture photos with expo-camera or pick from the gallery
- Upload images to Cloud Storage and get a download URL
- Store the URL on the post document (not a local path)
- Basics of storage security rules and pricing plans
- Different data models
- Introduction to Firebase Authentication



Plugins = bridges

Write JS; the plugin handles Swift/Kotlin under the hood.



Project step

Attach a photo (camera or gallery) to a new post.

The 12 sessions

1

Introduction & Foundations

2

Components & Navigation (Expo Router)

3

Components, State & Local Storage

4

Forms, Modals & Creating Data

5

Firebase Firestore CRUD

6

Firestore CRUD & Camera

7

Firebase Storage, Camera & Image Picker

8

Firebase Authentication & Context

9

Google sign-in, Firebase Rules & Guards

10

NativeWind & Geolocation

11

Google Maps & Queries

12

Exam & Wrap-Up

What is new in the codebase?

- We have added a new `imageApi.ts` under the `api` folder.
- We have updated the `ImageSelector.tsx` to integrate image picker in addition to capturing camera, as well as features to rotate, crop images.
- We have updated `index.tsx`, `handleNewPost` to support uploading images to Firebase Storage.
- We have updated data models in `postData.ts`.

SESSION 7 • STRUCTURE

How a 2-hour session runs

0:00-0:05	Recap & objectives Where we are in the project; today's goals
0:05-0:45	expo-camera + image-picker Capture & pick images
0:45-1:00	Break —
1:00-1:40	Storage security rules & pricing plan & Introduction to Firebase Authentication security & pricing
1:40-1:45	Assignment brief & Q&A This week's deliverable, then questions

Progress bar — likes toward a goal

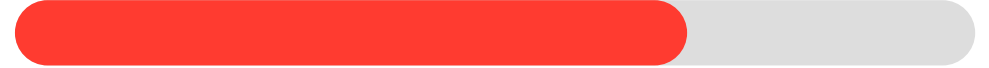
```
const LIKE_GOAL = 10;
const count = post.likes?.length ?? 0;
const progress = Math.min(count/LIKE_GOAL, 1);

<View style={track}>
  <View style={{
    width: `${progress * 100}%`,
    backgroundColor: progress >= 1
      ? "#34C759" : "#FF3B30",
  }} />
</View>
```

4 / 10 likes → red, in progress



7 / 10 likes → still red



10 / 10 likes → goal reached, turns green



Bar turns green when the goal is reached. No extra library needed.

Expo-camera & Expo-image-picker Recap

- Expo-camera is an Expo module that exposes camera functionality to React Native apps.

It provides following key features:

- CameraView component for preview
 - APIs such as `takePictureAsync()` for taking photos
 - Permission helpers for requesting camera access
 - A single JavaScript API across iOS/Android.
 - Important: expo-camera should be tested on a real device. It does not work on an iOS simulator
-
- Expo-image-picker is a high-level API to pick images from the photo gallery.
 - It can open the system image picker UI and is easier to test during development.

Two layers of Camera Permission in Expo

- Camera permissions in Expo requires two layers:
 - **Native permissions declaration (build time).**
 - Required for standalone/native builds. Configured through app.json and Expo config plugins.
 - The operating system must know in advance that your app intends to use the camera.
 - iOS: requires a usage string in Info.plist (e.g. NSCameraUsageDescription).
 - Android: requires camera permissions in AndroidManifest.xml. Expo automatically generates the appropriate native configuration when building the app.
 - With Expo Go, this native configuration is already included, so you do not need to add camera plugin.
 - **Runtime permission (when the feature is used).**
 - Still required in both Expo Go and your own native build.
 - Request permission in JavaScript, e.g., with `useCameraPermissions()` / `requestPermission()`.
 - Ask when the user actually opens the camera feature, not necessarily at app launch.
 - Handle granted / denied / restricted states gracefully.

Two layers of permission for Native

```
"ios": {
  "supportsTablet": true,
  "infoPlist": {
    "NSCameraUsageDescription": "We need access to your camera.",
    "NSPhotoLibraryUsageDescription": "We need access to your photos to select images.",
    "NSPhotoLibraryAddUsageDescription": "We save photos you take to your library.",
    "NSMicrophoneUsageDescription": "We use the microphone when recording video."
  }
},
"android": {
  "adaptiveIcon": {
    "backgroundColor": "#E6F4FE",
    "foregroundImage": "./assets/images/android-icon-foreground.png",
    "backgroundImage": "./assets/images/android-icon-background.png",
    "monochromeImage": "./assets/images/android-icon-monochrome.png"
  },
  "edgeToEdgeEnabled": true,
  "predictiveBackGestureEnabled": false,
  "permissions": [
    "CAMERA",
    "READ_MEDIA_IMAGES",
    "READ_MEDIA_VIDEO",
    "READ_EXTERNAL_STORAGE"
  ]
},
```

UX

Example of app.json

Two layers of permission - Native

```
"plugins": [  
  "expo-router",  
  [  
    "expo-image-picker",  
    {  
      "photosPermission": "The app accesses your photos to let you share them with your friends."  
    }  
  ],  
  [  
    "expo-camera",  
    {  
      "cameraPermission": "We need to access your camera so that you can take beautiful pictures!"  
    }  
  ],  
  [  
    ]  
  ]  
]
```

UX

Example of app.json

Permissions in Expo Go

- When running the project in Expo Go using **`npx expo start`**, no native project is actually generated. Metro bundler starts, Expo Go loads your JavaScript bundle, and your code runs inside the already-installed Expo Go app.
- Expo Go app is a compiled native application. It already contains:
 - Its own Info.plist
 - Its own AndroidManifest.xml
 - Supported native modules such as the camera module.
 - Required native permission declarations such as native camera support.
- Expo Go does not generate a new Info.plist or AndroidManifest.xml. However, you still need to request runtime permission in your code.

Two layers of permission recap

- **Development Notes**

- Web applications do not require native permission configuration in app.json; permissions are handled by the browser.
- Camera functionality is limited on iOS and Android simulators because simulators do not provide real camera hardware.
- For real devices and production builds, both static permissions (app.json) and runtime permission requests are required.
- However, the runtime permission check is still generally required for Web or simulators.

Capture picture from camera recap

- expo-camera opens the camera.
- Calling takePictureAsync() returns a uri you can render with <Image>.
- CameraView renders the camera preview.

camera permission hook

```
import { useCameraPermissions }
  from 'expo-camera';

const [permission, requestPermission] =
  useCameraPermissions();
if (! permission?.granted) requestPermission();
```

capture a photo

```
import { CameraView } from 'expo-camera';
import { useRef } from 'react';

const ref = useRef<CameraView>(null);

async function snap() {
  const photo = await ref.current?.takePictureAsync();
  setImageUri(photo?.uri);
}

<CameraView ref={ref} style={{ flex: 1 }} />
```

Pick from the photo library

- expo-image-picker opens the system gallery.
- Calling `launchImageLibraryAsync()` returns a uri you can render with `<Image>`.
- Same uri shape as the camera — unify both paths.

gallery permission hook

```
import { useMediaLibraryPermissions }
  from 'expo-image-picker';
const [mediaLibraryPermission,
requestMediaLibraryPermission] =
useMediaLibraryPermissions();

if (!mediaLibraryPermission?.granted) {
  const result = await requestMediaLibraryPermission();
  if (!result.granted) {
    return;
  }
}
```

launchImageLibraryAsync

```
import * as ImagePicker from 'expo-image-picker';

async function pickFromGallery() {
  const res = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ['images'],
    quality: 0.7,
  });
  if (!res.canceled) setImageUri(res.assets[0].uri);
}
```

Allow Multiple Images

- The code allows selecting multiple images.
- Images contains the currently selected image URIs.
- `allowsMultipleSelection: true` allows multiple image selected.
- The same logic applies to `takePhoto` function.
- `onChange()` appends the new image uri to **images**, and passes to the parent component before passing down again for showing.
- Why? Because images will also be used in the parent component `PostForm`.

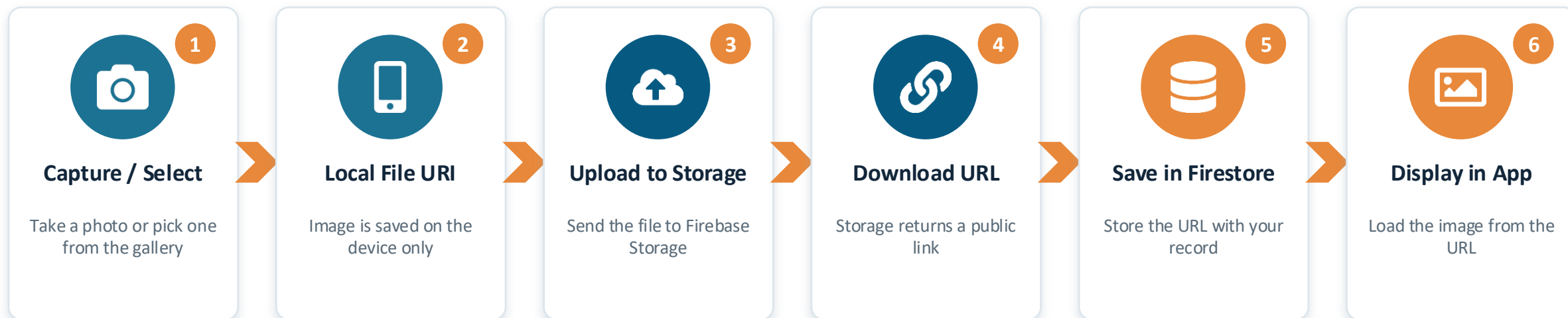
```
type ImageSelectorProps = {
  images: string[]; // array of local file URIs
  onChange: (images: string[]) => void;
};

export default function ImageSelector({ images, onChange }: ImageSelectorProps) {
  const [processing, setProcessing] = useState<number | null>(null); // index being processed

  // — Picker helpers —————

  async function pickFromGallery() {
    const { status } = await ImagePicker.requestMediaLibraryPermissionsAsync();
    if (status !== "granted") {
      Alert.alert("Permission required", "Allow gallery access in Settings.");
      return;
    }
    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ["images"],
      allowsMultipleSelection: true,
      quality: 0.85,
    });
    if (!result.canceled) {
      onChange([...images, ...result.assets.map((a) => a.uri)]);
    }
  }
}
```

From Camera to Cloud Storage



Why is the upload step needed?



Photos live on the device

A captured image is stored only in local storage at first.



Local files are private

Other users and devices can't reach files on your phone.



The file must travel

Uploading sends it across the network to a shared place.



Cloud makes it shareable

Stored in the cloud, the image is available to everyone.

Firestore Products



Realtime Database

Build serverless apps by storing and syncing JSON data between your users in near-realtime, on or offline, with strong user-based security.

[Learn more](#)



Cloud Functions

Write and run app logic server-side without needing to set up your own server.

[Learn more](#)



Authentication

Add an end-to-end identity solution to your app for easy user authentication, sign-in, and onboarding in just a few lines of code.

[Learn more](#)



Cloud Messaging

Get infrastructure to reliably send and receive push messages between your server and devices, across platforms at no cost.

[Learn more](#)



Hosting

Deploy fast-loading, secure websites that are backed by a global CDN without all of the hassle.

[Learn more](#)



Cloud Storage

Store and serve user-generated content with ease as your app grows from prototype to production-ready.

[Learn more](#)

Firestore stores data — Storage stores files

- **Cloud Storage holds large binary files: images, video, documents.**
 - After upload, you will get a download URL (<https://firebasestorage...>).
 - Save that URL on the Firestore document — never a file:// path.
 - Why? file:// paths are device-local.

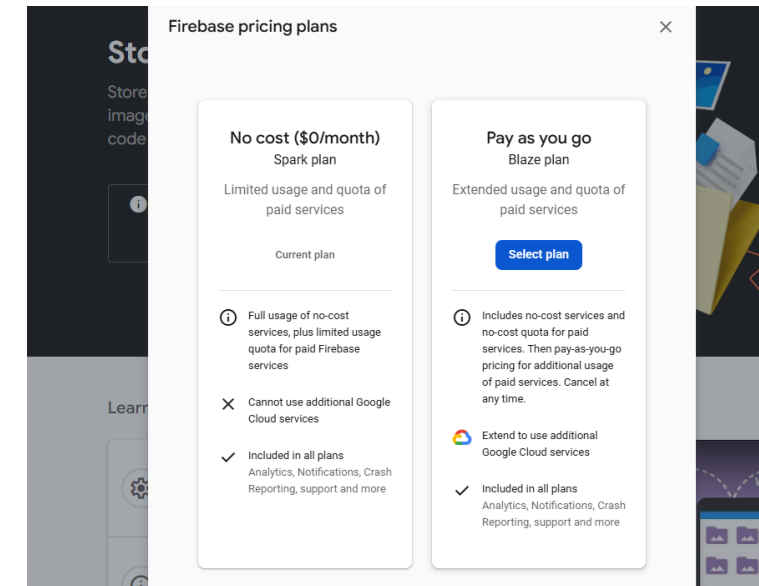
RULE

Storing the download URL makes images render correctly everywhere.

Firestore stores data — Storage stores files

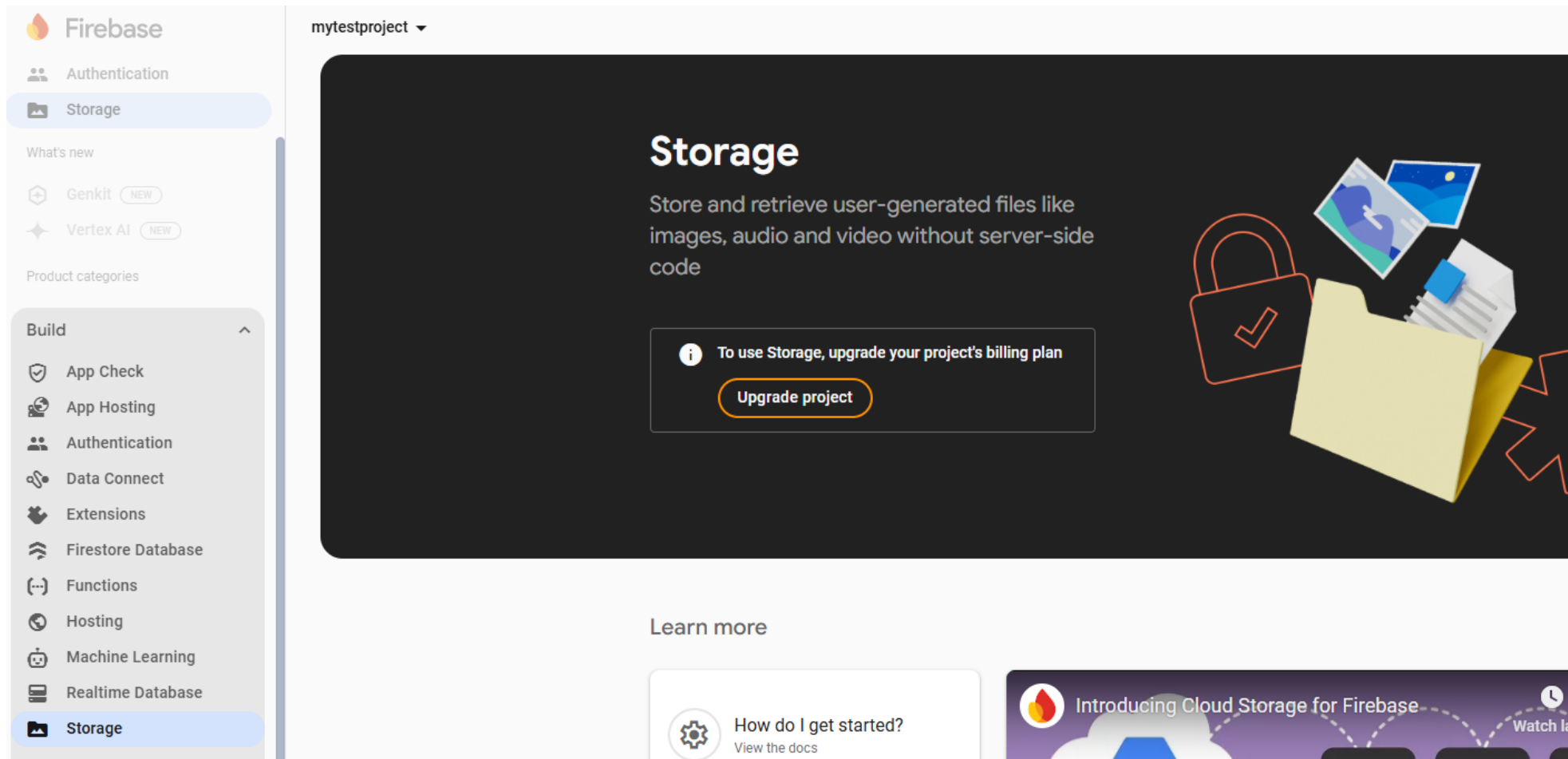
- Plans:

- As of 2026, Cloud Storage for Firebase requires the pay-as-you-go Blaze plan (billing enabled). This means a need to attach a billing account to use Firebase Storage.
- This does not necessarily mean you will actually be charged. Blaze still includes no-cost usage quotas. The free quota is usually far more than enough.
- For the newer *.firebasestorage.app buckets, no-cost allowances on Blaze is:
 - 5GB-months storage....
- Older Spark projects keep limited legacy access.
- <https://firebase.google.com/docs/projects/billing/firebase-pricing-plans>



Cloud Storage

- Add Cloud storage to a Firebase Project



The screenshot displays the Firebase console interface. On the left, a sidebar contains the 'Firebase' logo and a list of services: Authentication, Storage (highlighted), Genkit (NEW), and Vertex AI (NEW). Below these are 'Product categories' and a 'Build' section with various tools like App Check, App Hosting, Authentication, Data Connect, Extensions, Firestore Database, Functions, Hosting, Machine Learning, Realtime Database, and Storage (highlighted). The main content area is for 'mytestproject' and features a large dark card titled 'Storage'. This card includes the text 'Store and retrieve user-generated files like images, audio and video without server-side code' and a message: 'To use Storage, upgrade your project's billing plan' with an 'Upgrade project' button. An illustration of a folder with photos and a padlock is also present. At the bottom, there is a 'Learn more' section with a 'How do I get started?' link and a video thumbnail titled 'Introducing Cloud Storage for Firebase'.

Add a Collection

- You can manually create collection, upload file and delete file.
- However, you are encouraged to use Firebase storage SDK to interact with Firebase Storage programmatically.

The screenshot displays the Firebase Storage console interface. On the left, a sidebar contains navigation links for Authentication, Firestore Database, and Storage (which is selected). Below these are sections for 'What's new' (Genkit, Vertex AI), 'Product categories' (Build, Run, Analytics, AI), 'All products', and 'Related development tools' (IDX, Checks).

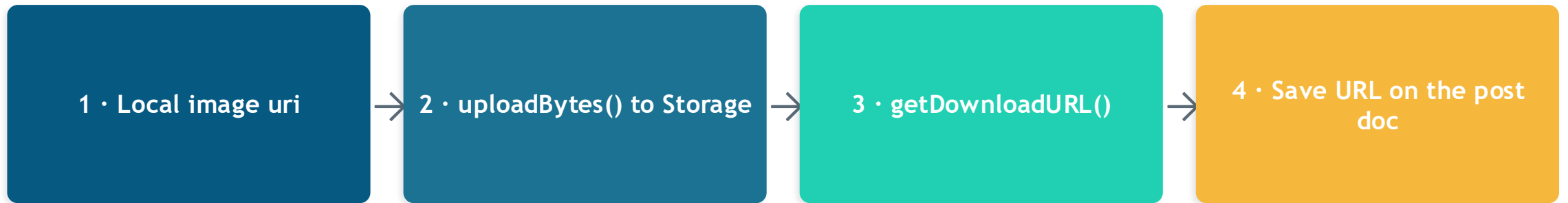
The main area shows a notification banner about pricing plan requirements. Below it, a modal dialog titled 'gs://my-awesome-project-704b5.appspot.com' is open, prompting the user to enter a 'Folder name' in a text input field. The dialog includes 'Upload file', 'Cancel', and 'Add folder' buttons.

Below the dialog, the storage view shows a table of contents:

Name	Size	Type	Last modified
images/	—	Folder	—
html-css.png	98.58 KB	image/png	10 Mar 2025

At the bottom, a blue action bar indicates '1 selected' item, with 'Download' and 'Delete' buttons. Below this bar, the same table is shown, but the 'html-css.png' file is selected with a checkmark.

The upload flow



Firestore never holds the bytes — only the URL of the images in Storage.

THEORY + FUNCTIONS

Upload an image

firebaseConfig.ts (add)

```
import { getStorage } from 'firebase/storage';  
export const storage = getStorage(app);
```

storage rule (read-all example)

```
service firebase.storage {  
  match /b/{bucket}/o {  
    match /{allPaths=**} {  
      allow read: if true;  
    } } }  
}
```

uploadImage()

```
import { ref, uploadBytes, getDownloadURL }  
  from 'firebase/storage';  
import { storage } from '../firebaseConfig';  
  
export async function uploadImage(uri: string) {  
  const blob = await (await fetch(uri)).blob();  
  const r = ref(storage, `posts/${Date.now()}.jpg`);  
  await uploadBytes(r, blob);  
  return getDownloadURL(r); // store THIS url  
}
```

THEORY + FUNCTIONS

Upload, then save the URL

create a post with a hosted image

```
// 1. upload each picked image, collecting download URLs.  
// .map() is a concise JS way to upload multiple images.  
const urls = await Promise.all(localUris.map(uploadImage));  
  
// we can use a simple for loop as well  
const urls: string[] = [];  
  
for (const uri of localUris) {  
  const url = await uploadImage(uri);  
  urls.push(url);  
}  
  
// 2. write the post with the hosted URLs  
await addDoc(collection(db, 'posts'), {  
  title, images: urls, createdAt: serverTimestamp(),  
});  
  
// 3. the feed's <Image source={{ uri }} /> now works on every device
```

THEORY + WORKFLOW

Saving a Post with Images

- In Postify app, you first call `uploadImage` to upload the selected image or captured photo to Firebase Storage, and wait for the returned url.
- You then build a new post object with your fields inputs and `imageUrls`.
- The last step is to call `createPost()` in `postApi` which creates a new document in Firestore `posts` collection.

```
// CREATE
export async function createPost(data: NewPost): Promise<void> {
  await addDoc(collection(db, POSTS_COLLECTION), {
    ...data,
    likes: [],
    comments: [],
    imageUrls: data.imageUrls ?? [],
    createdAt: serverTimestamp(),
  });
}
```

```
async function handleNewPost(post: PostData, localImages: string[]) {
  try {
    let imageUrls: string[] = [];

    if (localImages.length > 0) {
      Toast.show({
        type: "info",
        text1: `Uploading ${localImages.length} image${localImages.length > 1 ? "s" : ""}...`,
      });
      const uploadId = Date.now().toString();
      for (let i = 0; i < localImages.length; i++) {
        const url = await uploadImage(localImages[i], uploadId, i);
        imageUrls.push(url);
      }
    }

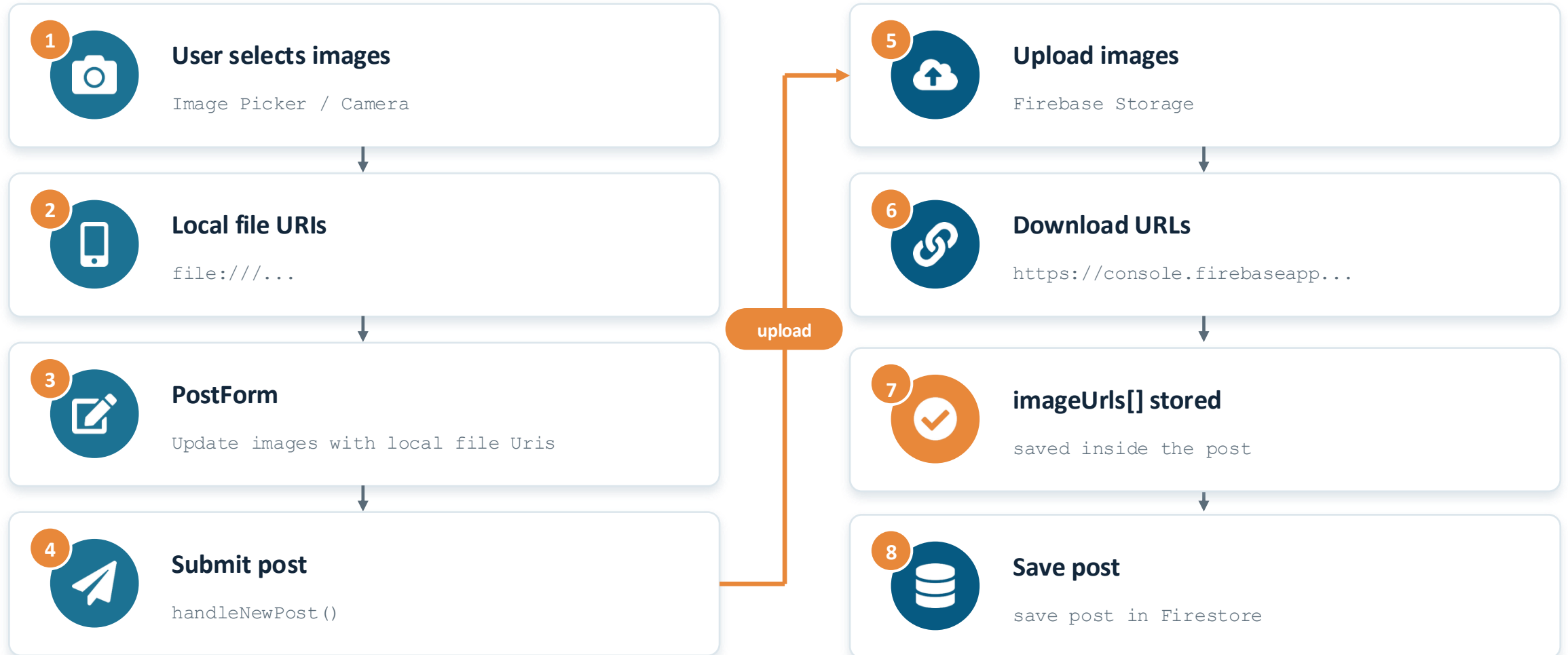
    await createPost({
      title: post.title,
      description: post.description,
      hashtags: post.hashtags,
      author: post.author,
      imageUrls,
    });

    Toast.show({
      type: "success",
      text1: imageUrls.length > 0 ? "Post with images saved!" : "Post saved!",
    });
    await loadPosts();
  } catch (error) {
    console.error("Failed to save post:", error);
    Toast.show({
      type: "error",
      text1: "Could not save post.",
      text2:
        error instanceof Error ? error.message : "Check your connection.",
    });
  }
}
```

Uploading Images to Cloud

 On the device · steps 1–4

 In the cloud · steps 5–8



Loading Images from Cloud



Loading Images from Cloud

- The homePage only shows the first image of each post when multiple images are available.
- The postDetails page shows all images using map to iterate.

```
<TouchableOpacity>
  { /* Show first image as card header if available */
  {firstImage && (
    <Image source={{ uri: firstImage }} style={styles.cardImage} resizeMode="cover" />
  )}
  <View style={styles.body}>
    <Text style={styles.title}>{postData.title}</Text>
    <Text style={styles.description}>{postData.description}</Text>
  </View>
</TouchableOpacity>
```

```
{ /* — Image gallery — */
{ /* These are Firebase Storage download URLs, so they load on every device. */
{imageUrls.length > 0 && (
  <ScrollView
    horizontal
    showsHorizontalScrollIndicator={false}
    style={styles.imageScroll}
    contentContainerStyle={styles.imageScrollContent}
  >
    {imageUrls.map((url, i) => (
      <Image
        key={i}
        source={{ uri: url }}
        style={styles.detailImage}
        resizeMode="cover"
      />
    ))}
  </ScrollView>
)}
)}
```

Firestore & Storage

- Firebase Storage stores the actual image files.
- Firestore stores only metadata about the post.
- `imageUrls` contains the download URLs returned by Firebase Storage after upload.
- When rendering a post, the app loads images using those URLs.
- This keeps Firestore documents small and efficient. It also allows the same image to be accessed from multiple devices.

Two Post data models

- Why do we define two different data models - NewPostData and PostData?
- addDoc generates the document id server-side - Firestore assigns it as the document's key, not as a field inside the document body.
- NewPostData describes what is actually written to FireStore.
- createPost in postApi builds PostData from partial NewPostData (connected to UI input). It initialize the rest: likes, comments, createdAt. User interactions later update likes, comments.
- In NewPostData, likes, comments, imageUrls, are optional. Therefore when we create a new post from PostForm, we do not have to specify partial.

```
export interface NewPostData {  
  title: string;  
  description: string;  
  hashtags: string;  
  author: string;  
  likes?: string[];  
  comments?: string[];  
  imageUrls?: string[]; // Firebase Storage download URLs, set after upload  
}  
  
export interface PostData extends NewPostData {  
  id: string;  
}
```

```
const post: NewPostData = {  
  title: title.trim(),  
  description: description.trim(),  
  hashtags: chips.map((t) => `#${t}`).join(" "),  
  author: authorName || "Anonymous",  
};  
onSubmit(post, images);
```

Firestore Products



Realtime Database

Build serverless apps by storing and syncing JSON data between your users in near-realtime, on or offline, with strong user-based security.

[Learn more](#)



Cloud Functions

Write and run app logic server-side without needing to set up your own server.

[Learn more](#)



Authentication

Add an end-to-end identity solution to your app for easy user authentication, sign-in, and onboarding in just a few lines of code.

[Learn more](#)



Cloud Messaging

Get infrastructure to reliably send and receive push messages between your server and devices, across platforms at no cost.

[Learn more](#)



Hosting

Deploy fast-loading, secure websites that are backed by a global CDN without all of the hassle.

[Learn more](#)



Cloud Storage

Store and serve user-generated content with ease as your app grows from prototype to production-ready.

[Learn more](#)

What Firebase Auth handles for you

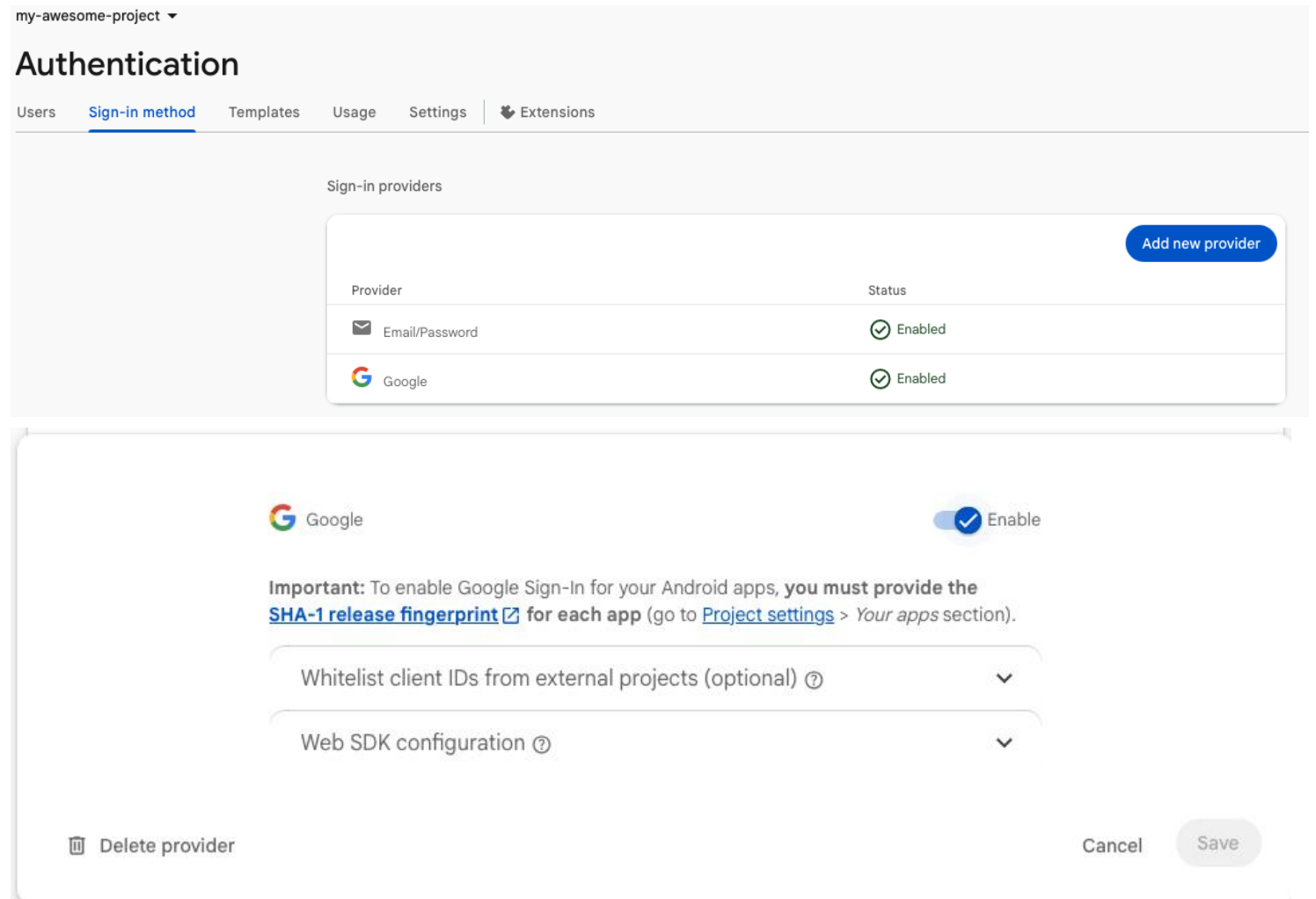
- **Securely stores credentials and issues a session token.**
 - Many providers: email/password, Google, Apple, GitHub, anonymous...
 - Each signed-in user gets a unique uid — your key for ownership.
 - `onAuthStateChanged` tells the app when sign-in state changes.
- **OAuth 2.0 (Google): the user authenticates with Google, not you.**
 - You never see their password; you receive a verified identity.
 - We will introduce Google sign-in in session 9.

SECURITY

Never store raw passwords yourself.
Let the provider do it.

What Firebase Auth handles for you

- In Sign-in method, you can enable/disable method.



SETUP + AUTHENTICATION

Firebase Auth Persistence

- You need to configure firebaseConfig.ts for Firebase Authentication, just as Firebase Firestore, and Firebase Storage.
- However, we initialize Firebase Auth differently.
- Web uses `getAuth(app)`, since Firebase can use browser persistence, and login can survive page refreshes.
- For native React Native apps (including iOS simulator, Android Emulator, and physical devices), there is no browser `localStorage`. It uses `AsyncStorage` for persistent login.
- It therefore requires `initializeAuth()`.
- We will talk about Email/Password Authentication in next session.

firebaseConfig.ts

```
import { getAuth } from 'firebase/auth';  
export const auth = getAuth(app);
```

```
// Web uses getAuth (defaults to browserLocalPersistence).  
// Native uses initializeAuth with AsyncStorage so the session survives app restarts.  
// require() is used so Metro only loads firebase/auth/react-native on native bundles.  
function createAuth() {  
  if (Platform.OS === "web") {  
    return getAuth(app);  
  }  
  // eslint-disable-next-line @typescript-eslint/no-var-requires  
  const { getReactNativePersistence } = require("@firebase/auth/dist/rn/index.js");  
  return initializeAuth(app, {  
    persistence: getReactNativePersistence(AsyncStorage),  
  });  
}  
  
export const auth = createAuth();
```



Make ImageSelector a Modal

YOUR TASKS

Build a reusable SelectImageModal component that shows a live camera preview full screen, lets the user pick from the gallery or snap a photo, and returns the image URI to the caller.

This introduces:

- `CameraView` as a fullscreen background preview
- `useCameraPermissions` — handling the three permission states (loading, denied, granted)
- Simulator detection with `expo-device`
- Passing callbacks as props (`closeModal`, `setImage`)
- Wiring a modal component into an existing screen

DELIVERABLE

A reusable SelectImageModal component that can open the camera, take a photo, pick an image from the gallery, and return the selected image URI to the parent screen.

EST. EFFORT

≈ 4 hours